COMP3320/6464 Week 3, Lecture 1

Rhys Hawkins

Semester 2, 2024

Course Representatives

COMP3320

- Chengbo Yan (u7515796@anu.edu.au)

COMP6464

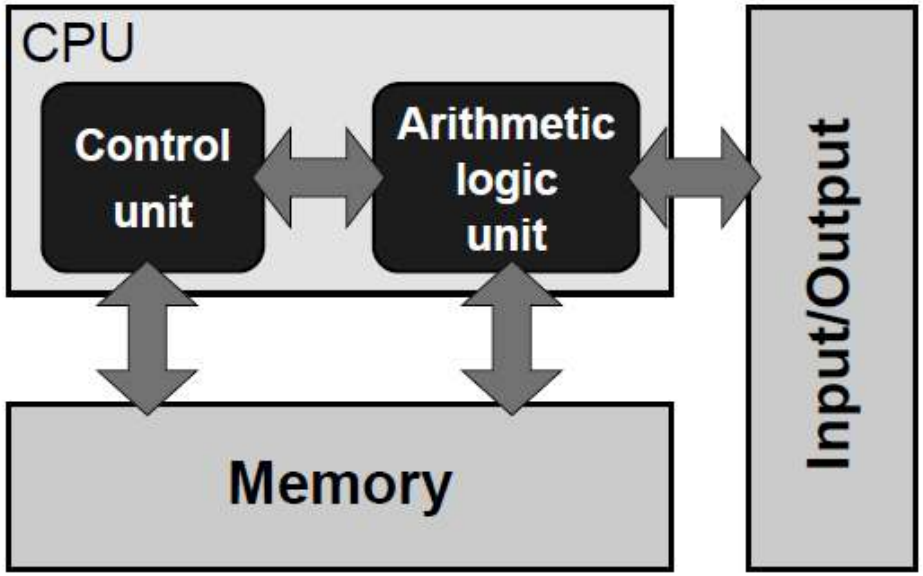
- Hailong Gong (Hailong.Gong@anu.edu.au)

Previously

- Some simple/general optimisation techniques
- That sometimes don't (appear) to work
- Benchmark/Test always

von Neumann Computer Architecture

Figure 1.1: Stored-program computer architectural concept. The "program," which feeds the control unit, is stored in memory together with any data the arithmetic unit requires.



- instructions and data continuously fed
- inherently sequential (SISD)

Harware techniques to improve performance

- Simplified instruction set
- Pipelined functional units
- Super scalar architecture
- Super-pipelining
- Larger caches
- Out-of-order execution
- Data parallelism through SIMD instructions

Instruction Set Architectures

- Early microprocessors were very simple, but in 1964 IBM introduced the 360 series which was microprogrammed.
- From then instruction sets and addressing modes increased, prompted in part by development of high level languages.
- Special microcode was added to handle case statements, procedure calling, array indexing etc.
  - led to the CISC concept (Complex Instruction Set Computer)
- In the 70s writing, debugging and maintaining microcode became a major issue.
- Academics begin to analyse what programs actually did and this resulted in a major rethink of microprocessor design
  - led to the RISC concept (Reduced Instruction Set Computer)

Typical Program Instructions (late 70's)

Statement	SAL	XPL	Fortran	C	Pascal	Average
Assignment	47	55	51	38	45	47
If	17	17	10	43	29	23
Call	25	17	5	12	15	15
Loop	6	5	9	3	5	6
Goto	0	1	9	3	0	3
Other	5	5	16	1	6	7

Conclusion: most programs consist of assignments, if statements and procedure calls

Typical Assigments and Procedures (late 70's)

Assignment	
N Terms	
0	-
1	80
2	15
3	3
4	2
≥ 5	0

Procedures

N Locals	
0	22
1	17
2	20
3	14
4	8
≥ 5	20

N Parameters	
0	41
1	19
2	15
3	9
4	7
≥ 5	8

- 80% of all assigments are variable = value
- 21% of all procedures have no local variables
- 41% of all procedures have no parameters

Theoretically people can write highly complex programs, but the ones that they do write are actually very simple consisting of assignments, if statements and procedure calls with limited numbers of arguments

Comparison of CISC and early RISC machines

	CISC			RISC		
	IBM 370/168	VAX 11/780	Xerox Dorado	IBM 801	Berkeley RISC1	Stanford MIPS
Year Completed	1973	1978	1978	1980	1981	1983
Instructions	208	303	270	120	39	55
Microcode size (bytes)	54K	61K	17K	0	0	0
Instruction size (bytes)	2-6	2-57	1-3	4	4	4
Execution Model	Reg-reg Reg-mem mem-mem	Reg-reg Reg-mem mem-mem	Stack	Reg-reg	Reg-reg	Reg-reg

Load/Store Architecture

- Memory reference restricted to load/store
  - Only one reference per instruction
  - In CISC arithmetic/logical instruction may include memory reference
- Motivation:
  - To enable fixed instruction length
  - To ease pipelining
  - Since memory reference may be slow

Uniform Instruction Length

- Fundamental RISC feature
- Eases decomposition into pipeline stages
- CISC machine had no a priori knowledge of length of each instruction

Characteristics of RISC and CISC machines

	RISC	CISC
1	Simple instructions taking 1 cycle	Complex instructions taking multiple cycles
2	Only LOADS/STORES reference memory	Any instruction may reference memory
3	Highly pipelined	Not pipelined or less pipelined
4	Instructions executed by the hardware	Instructions interpreted by the microcode
5	Fixed format instructions	Variable format instructions
6	Few instructions and modes	Many instructions and modes
7	Complexity is in the compiler	Complexity is in the microprogram
8	Multiple register sets	Single register set

- Assembly language programmers used the complicated machine instructions, but compilers generally did not. Difficult to get compiler to recognise complicated instructions.
- Optimising compiler work by simplifying and eliminating redundant computations. After a pass through optimizing compiler, opportunities to use the complicated instructions tend to disappear.

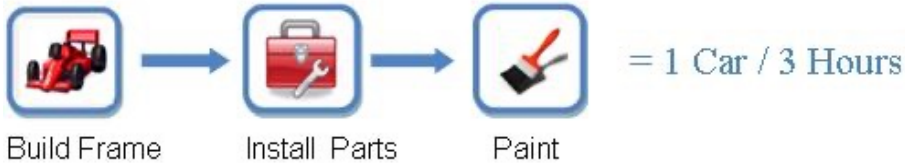
RISC Processors

- First Generation Characteristics
  - Pipelining (both instruction and floating point)
  - Branching (delayed branching and branch prediction)
  - Uniform instruction length
  - Load/Store architecture (simple addressing)
- Second Generation
  - Faster clocks
  - Superpipelining
  - Superscalar
- Post-RISC
  - Out-of-order execution

Today

- Pipelining
- Super Scalar Architecture
- Super Pipelining

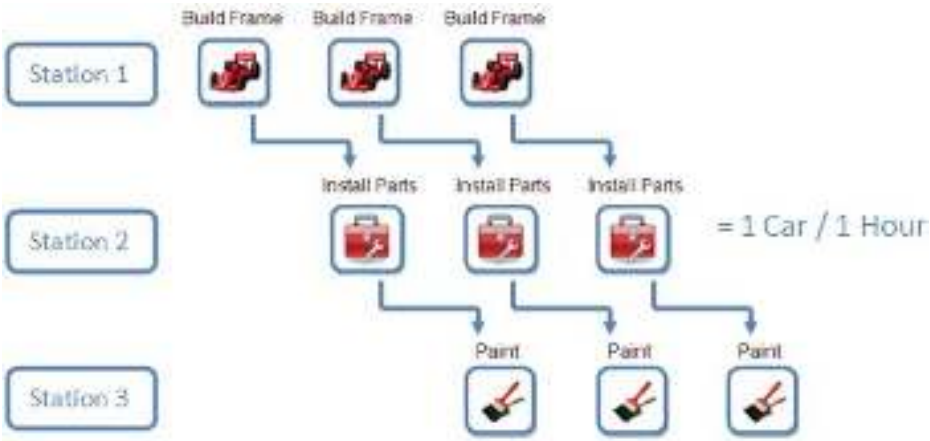
Pipelining: Car Manufacturing Analogy



$T_{seq} \equiv$ Time of sequential execution

$=$ 3 Hours

Pipelining: Car Manufacturing Analogy



$T_{pipe} \equiv$ Time of pipelined execution
$= 1$ Hour

$m \equiv$ Pipeline depth (number of stations)
$= 3$

Performance evaluation of a pipeline

- m = depth of pipeline
- N = number of items to process
- T_{seq} = time of sequential execution
- T_{pipe} = time of pipelined execution

$$\text{Speed-up} \equiv \frac{T_{seq}}{T_{pipe}} = \frac{mN}{N+m-1}$$

$$\text{Throughput} \equiv \frac{N}{T_{pipe}} = \frac{1}{1+\frac{m-1}{N}}$$

- For large N, speed up $\approx m$
- Throughput is efficiency of pipeline

Pipeline throughput

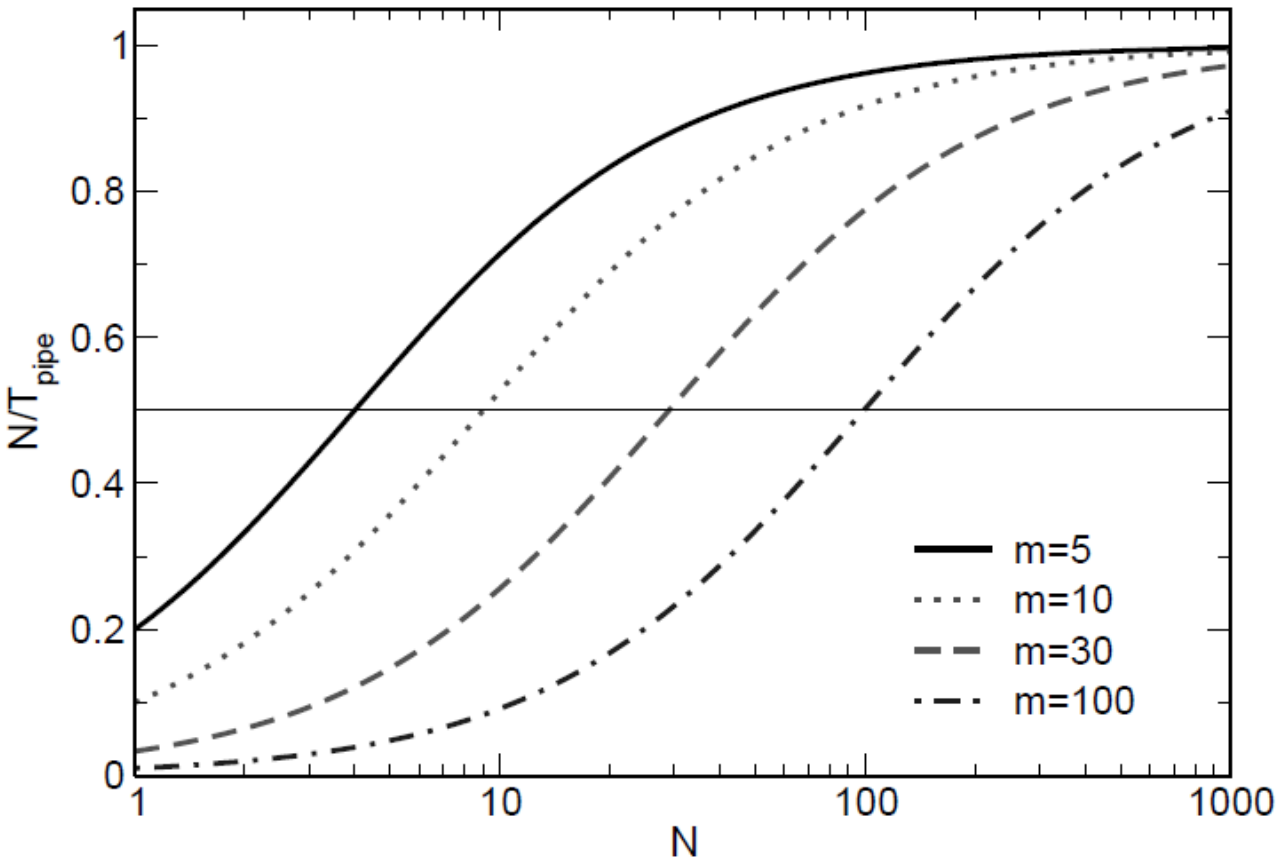


Figure 1.6: Pipeline throughput as a function of the number of independent operations. *m* is the pipeline depth.

Pipelining

- Break instruction execution into k stages
- can get k-way parallelism
- Generally, the circuitry for each stage is independent
- Stages:
  - FI = Fetch Instrn.,
  - DI = Decode Instrn.,
  - FO = Fetch Operand,
  - EX = Execute Instrn.,
  - WB = Write Back
- Independent instructions can be pipelined automatically (CPU and/or compiler)
- FO and WB stages may involve memory access (and may possibly stall the pipeline)

Pipelining: Dependent instructions

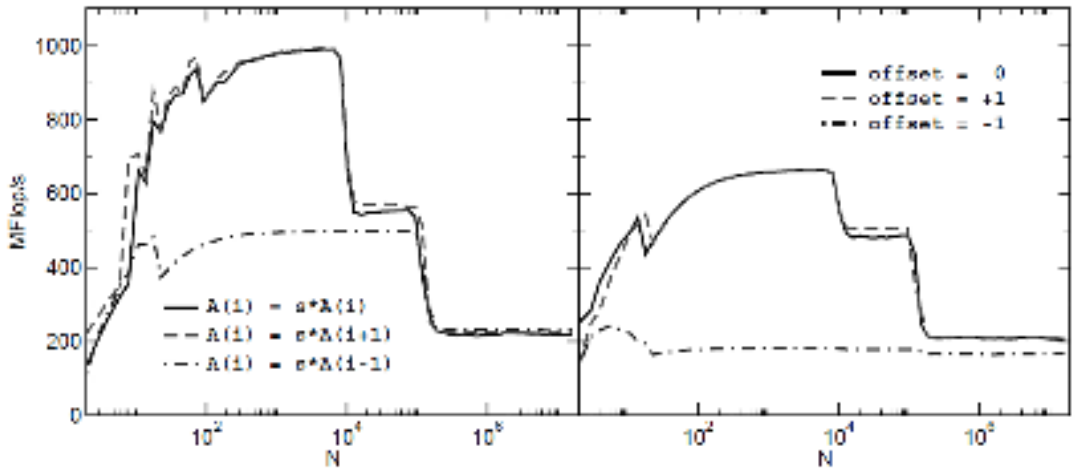


Figure 1.7: Influence of constant (left) and variable (right) offsets on the performance of a scaling loop. (AMD Opteron 2.0 GHz).

```
void iterate(int N, float *A, float s, int offset)
{
  int i;
  for (i = 0; i < N; i ++) {
    A[i] = s * A[i + offset];
  }
}
```

Pipelining: Dependent instructions

- Imagine a CPU with 3 independent pipelines and every operation takes one cycle
- `offset = 0`
- Each iteration independent

```
void iterate(int N, float *A, float s, int offset)
{
  int i;
  for (i = 0; i < N; i ++) {
    A[i] = s * A[i + offset];
  }
}
```

Cycles	Pipelines		
	$i = 0$	$i = 1$	$i = 2$
1	Load A[0]		
2	fmul s	Load A[1]	
3	Store A[0]	fmul s	Load A[2]
4		Store A[1]	fmul s
5			Store A[2]

Pipelining: Dependent instructions

- Imagine a CPU with 3 independent pipelines and every operation takes one cycle
- `offset = 1`
- Iterations dependent but can still be pipelined

```
void iterate(int N, float *A, float s, int offset)
{
  int i;
  for (i = 0; i < N; i ++) {
    A[i] = s * A[i + offset];
  }
}
```

Cycles	Pipelines		
	$i = 0$	$i = 1$	$i = 2$
1	Load A[1]		
2	fmul s	Load A[2]	
3	Store A[0]	fmul s	Load A[3]
4		Store A[1]	fmul s
5			Store A[2]

Pipelining: Dependent instructions

- Imagine a CPU with 3 independent pipelines and every operation takes one cycle
- `offset = -1`
- Each iteration dependent

```
void iterate(int N, float *A, float s, int offset)
{
  int i;
  for (i = 0; i < N; i ++) {
    A[i] = s * A[i + offset];
  }
}
```

Cycles	Pipelines		
	$i = 1$	$i = 2$	$i = 3$
1	Load A[0]		
2	fmul s	*wait*	
3	Store A[1]	*wait*	*wait*
4		Load A[1]	*wait*
5		fmul s	*wait*
6		Store A[2]	*wait*
7			Load A[2]
8			fmul s
9			Store A[3]

Pipelining: Dependent instructions

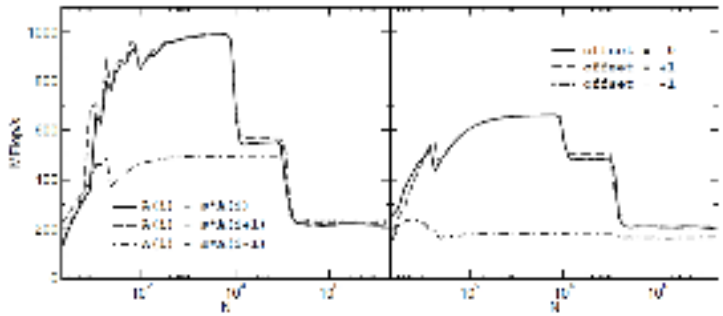


Figure 1.7: Influence of constant (left) and variable (right) offsets on the performance of a scalar loop (AMD Opteron 2.0 GHz).

Summary

- CPU must ensure same result (pipelined or not)
- Dependent instructions must be delayed (No-ops)
- Modern CPUs have different pipelines optimised for different operations
- Many instructions in practice have latencies of a few cycles (FP *, +) to 10s of cycles (FP /)

Pipelining: How to deal with dependent instructions

- with `offset = -1`

```
void iterate(int N, float *A, float s)
{
  int i;
  for (i = 1; i < N; i ++) {
    A[i] = s * A[i - 1];
  }
}
```

- First step: recognize that the result of the loop is A[0], s*A[0], s*s*A[0] etc

```
void iterate(int N, float *A, float s)
{
  int i;
  float p = s * A[0];
  for (i = 1; i < N; i ++) {
    A[i] = p;
    p = p * s;
  }
}
```

Pipelining: How to deal with dependent instructions

```
void iterate(int N, float *A, float s)
{
  int i;
  float p = s * A[0];
  for (i = 1; i < N; i ++) {
    A[i] = p;
    p = p * s;
  }
}
```

Cycles	Pipelines		
	$i = 1$	$i = 2$	$i = 3$
1	Store A[1] = p		
2	fmul p = s * p	*wait*	
3		Store A[2] = p	*wait*
4		fmul p = s * p	*wait*
5			Store A[3] = p
6			fmul p = s * p

- Marginal improvement but we still have a loop dependency
- Loop unroll

Pipelining: How to deal with dependent instructions

```
void n2loop(int N, float s, float *A)
{
  float p1 = A[0] * s;
  float p2 = p1 * s;
  float s2 = s*s;

  for (int i = 1; i < N; i += 2) {
    A[i] = p1;
    p1 = p1 * s2;

    A[i + 1] = p2;
    p2 = p2 * s2;
  }
}
```

Cycles		Pipelines	
	$i = 1$	$i = 2$	$i = 3$
1	Store A[1] = p1		
2	fmul p1 = p1*s2	Store A[2] = p2	
3		fmul p2 = p2*s2	Store A[3] = p1
4			fmul p1 = p1 s2

- No more waiting

Pipelining: How to deal with dependent instructions

Results from my laptop

Test	Time (μs)
`offset = 0` No Loop Unroll	1500
`offset = 0` x2 Loop Unroll	1300
`offset = 0` x4 Loop Unroll	1300
`offset = -1` Original No Loop Unroll	4750
`offset = -1` Original x4 Loop Unroll	4750
`offset = -1` Improved No Loop Unroll	4750
`offset = -1` Improved x2 Loop Unroll	2450
`offset = -1` Improved x4 Loop Unroll	1260

- Loop unrolling of `offset = 0` has little effect. Pipelining working automatically.
- Loop unrolling of `offset = -1` original code has no effect. Pipeline(s) have to wait.
- With loop unrolling and auxilliary variables we can get comparable performance
- Caution: auxiliary variables can introduce floating point errors

Pipelining: Branch instructions

- Branch to a new program address (perhaps caused by an if statement) will disrupt the pipeline flow.
- Processor doesn't know if instruction is a branch until the decode stage and then may not know if it will be taken until the execute stage.
- If branch is taken then following "in flight" instructions must be annulled.
- Enables the pipeline to continue for unconditional branches (i.e. when the decoded instructions says branch somewhere and we go there).
- Conditional branches are more difficult, pipeline will stall and require flushing as it can't be recognized before the DI stage

Pipelining: Branch prediction

To handle conditional branches various branch prediction schemes are used:

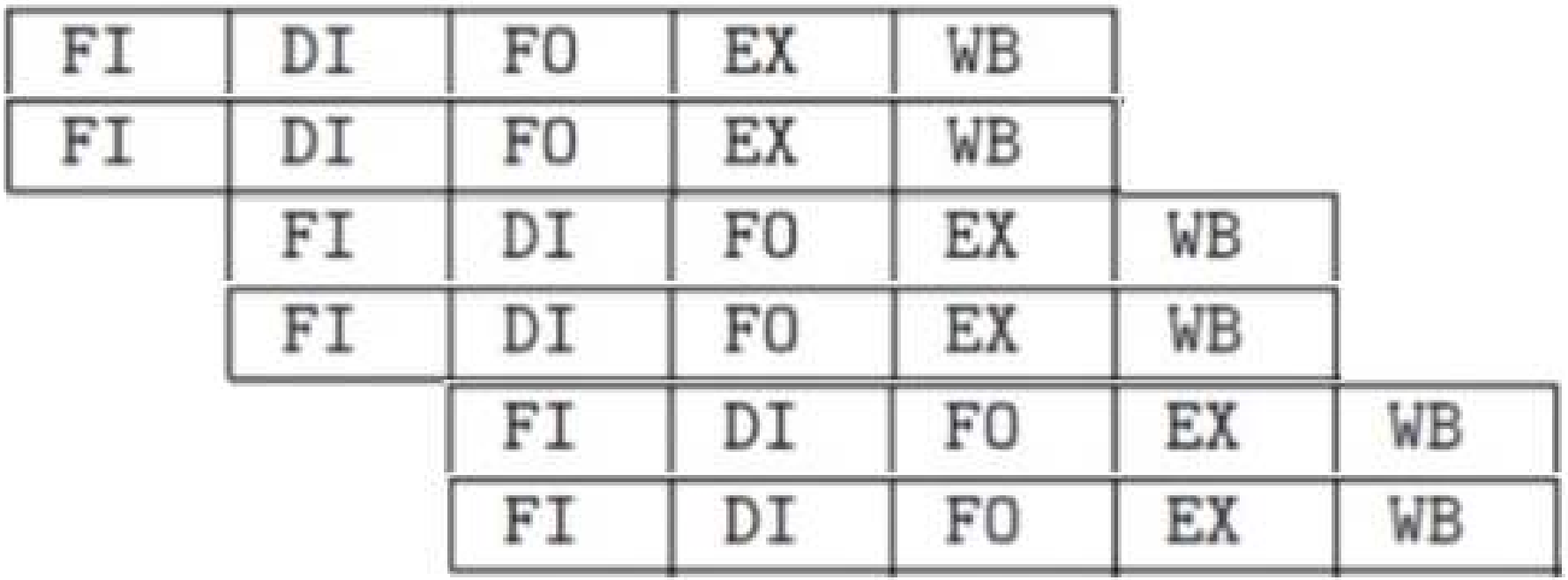
- Assume branches are always taken (flush pipeline when not taken)
  - OK for loops with test at bottom
- S/W (compiler) indicates the 'most likely' prediction
  - profile feedback-directed optimization
- H/W keeps a branch prediction buffer: predict using result of the last (few) executions of the branch

Summary: Pipelining and FP operations

- FP operations typically take longer than fixed point operations so benefit greatly from pipelining.
- Examples from the slides of nearly 4x speedup
- Number of stages in the pipeline may be increased so even complicated operations like FP * can be pipelined.
- FP +, -, *, comparison and conversion pipelined
- Usually sqrt and / are NOT pipelined
- Some processors limit overlap of FP operations due to shared internal components
- Fully pipelined implies no overlap restrictions
- Pipelining, removing loop dependencies and loop unrolling can cause bugs (always test)
- Floating point accuracy can be impacted by introducing, e.g. auxilliary accumulators (always test)
- We will discuss some other problems with loop unrolling in future lectures (aliasing)

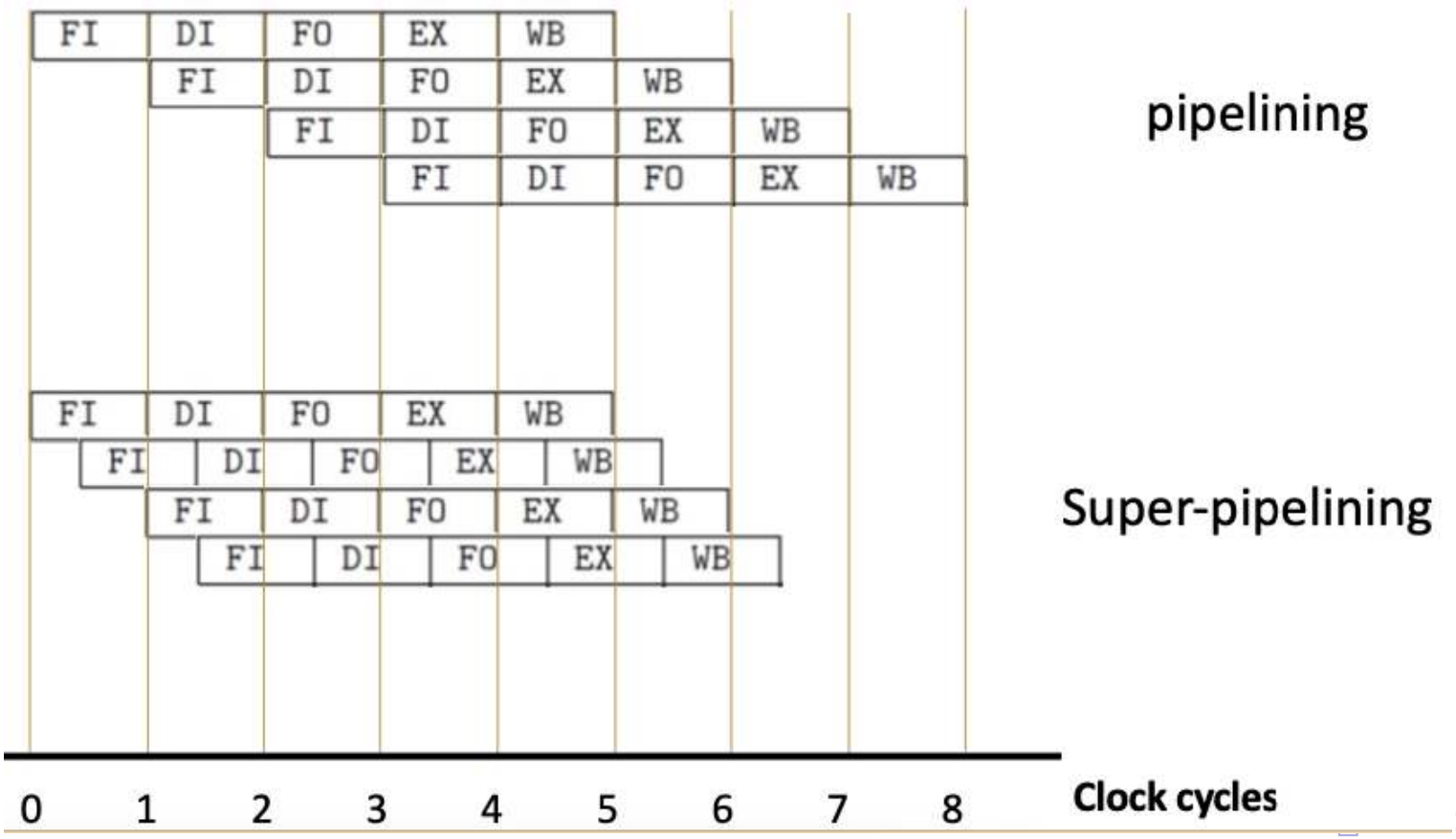
Superscalar Architecture

- Since around 1987
- Executes multiple independent instructions in parallel
- Common instructions (load/store +,*) can be initiated and executed independently



Super pipelining

- Since around 1988
- Many pipeline stages need (less than) half clock cycle
- Super-pipelining is breaking instructions into smaller stages
  - making pipeline deeper
  - enhancing instruction throughput



Out of Order Execution

- Arguments to instructions may not be available in registers ‘on time’ (memory vs processor speed)
- Excecute instructions that appear later in the instruction stream but have their parameters available.
- Avoids Idle times (stalls)
- Improves throughput
- Easier for compilers to arrange machine code
- Use a reorder buffer that stores instructions until they become eligible for execution
- Sometimes a CPU/Compiler does this for you. Other times you may need to reorder instructions yourself to improve performance.